

Terraform in CI/CD

Automate everything — scanning on pull requests, disposable environments, and drift control in a pipeline.

Validation & scanning

Checkov

PR workflow

Disposable envs

Lab 10

Today's roadmap



1 • Validate & scan

terraform validate + security scanning with Checkov.



2 • The PR workflow

Plan + scan on every pull request as a quality gate.



3 • Disposable envs & drift

Spin up, tear down, and keep infra honest.



4 • Lab 10 + finale

Build the PR workflow; watch the full CD run.

01

Validation & Scanning

Your infra is code — so scan it like code.

Code validation & scanning

Now that infra is code, add automated checks before anything is applied.

- `terraform validate` — syntax & provider checks
- Security scanners catch misconfigurations early
- Fail the build before insecure infra ships

✓ Scan the **code** and the **plan file** — catch what's declared and what will actually change.

Checkov

- Supports Terraform, ARM, Bicep, CloudFormation, Helm...
- Checks CIS, NIST & Well-Architected best practices
- Flags weak encryption, public exposure, permissive rules
- Custom checks & easy CI/CD integration

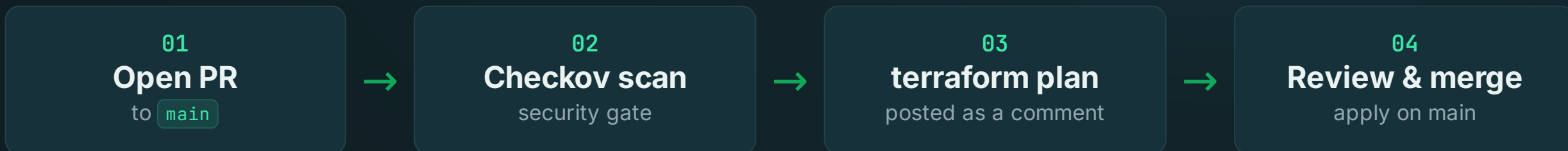
BASH

```
# scan code
checkov -d .

# scan a plan file
terraform show -json tf.plan > tf.json
checkov -f tf.json
```

⚠ Don't over-tighten — overly strict policies can block delivery and drive up cost. Tune the ruleset.

The Pull Request workflow



🛡️ Branch protection + secretless OIDC to Azure = a safe, auditable path to production.

A pull-request workflow

.GITHUB/WORKFLOWS/PR.YML

```
name: PR checks
on:
  pull_request:
    branches: [main]

permissions:
  id-token: write      # OIDC login to Azure
  pull-requests: write # post the plan comment

jobs:
  plan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - uses: azure/login@v2          # secretless
      - run: terraform init
      - run: checkov -d .              # security gate
      - run: terraform plan -no-color
```

- **Triggers** on every PR to `main` — the quality gate
- `id-token: write` enables **OIDC** — no stored credentials
- `checkov -d .` fails the build on insecure config
- `plan` output is posted back to the PR for review

💡 Apply runs on a separate `push: main` workflow — never on the PR.

02

Environments & Drift

Provision on demand — and never let reality drift
unseen.

Disposable environments

Create an environment only when needed, then delete it when the job is done.

- Optimize cost — nothing runs 24×7 for no reason
- Shrink the attack surface
- Give everyone their own throwaway env

create

apply on demand



test / use

run the job



destroy

tear it all down

Configuration drift in CI/CD

You provision infra once — every run after that **validates** it still matches your code.

- Idempotence solves drift out of the box
- A scheduled plan detects manual changes
- Fail the pipeline, or auto-sync

⇒ A drift check in CI is a **quality gate** that keeps your environments matching the code that describes them.

 LIVE DEMO

Checkov & the PR pipeline

Run Checkov locally, then watch the PR workflow scan the code and post the Terraform plan as a comment.

Demo · 10 **CI** · GitHub Actions

🔗 LAB 10 • 30-40 MIN

Terraform on Pull Requests

Add branch protection, wire OIDC to Azure, and create a PR workflow that runs Checkov + `plan` on your own repo.

Guided finale: the full CD workflow — environments, approvals & drift — runs as an instructor-led walkthrough to replay after class.

What you've built in 20 hours



From a repo...

Terraform created your Git repo on day one.



...to Azure infra

Network, App Services, database — modularized.



...to a pipeline

Scanned, reviewed and delivered via CI/CD.



Where next: workspaces, HCP Terraform / Terraform Cloud, policy-as-code (OPA / Sentinel), and `terraform test`.



Thank you

You're now ready to build real infrastructure with Terraform. Keep practising — and keep it in version control.